

Question Bank — Week 1

Why this course · Python basics (Lecture 01, Lab 01) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Questions: *week01-questions.md*. Read an answer only after you have committed to your own. For multiple choice, the explanation of why each wrong option is wrong is the part worth studying: the wrong options are the mistakes students actually make.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	b	M13	b	M19	c
M02	d	M08	a	M14	c	M20	a
M03	c	M09	c	M15	d	M21	d
M04	c	M10	b	M16	a	M22	b
M05	a	M11	d	M17	c	M23	c
M06	d	M12	a	M18	b	M24	d

W1-M01 — b. Turing-completeness is about *what can be computed*: the same set of problems. It says nothing about speed (a), is not the same claim as “only the syntax differs” (c) — languages differ in cost to write, read and run — and programs are certainly not portable between languages (d).

W1-M02 — d. *Dynamic*: types are checked at run time and a name can be rebound to a different type. *Strong*: Python refuses to mix incompatible types silently (see M03). “Static” (a, b) describes C, Java, Rust; “weak” (b, c) describes C’s implicit conversions or JavaScript’s `"3" + 5`.

W1-M03 — c. `TypeError: can only concatenate str (not "int") to str`. “35” (a) is JavaScript’s answer — weak typing. 8 (b) would need an implicit conversion Python refuses to make. It is valid syntax, so not (d).

W1-M04 — c. Ownership and borrowing are checked at compile time. Manual free (a) is C/C++; a garbage collector (b) is Python, Java, Go; (d) describes no mainstream language.

W1-M05 — a. Ritchie, Bell Labs, early 1970s, grown out of Thompson’s B, in order to write Unix (rewritten in C in 1973). Stroustrup (b) created C++; Knuth (c) created TeX; van Rossum (d) created Python.

W1-M06 — d. The ADT is the *contract*. Memory layout (a) is exactly what it leaves out — that belongs to the data structure. The language (b) and class hierarchy (c) are implementation choices.

W1-M07 — b. A generator expression inside `sum` states the result wanted. (a) and (d) are procedural — steps and state; (c) is functional — composing `map` and `filter`.

W1-M08 — a. TIOBE counts search-engine hits. It measures how much people write about a language, not quality (b); GitHub activity is Octoverse (c); self-reported usage is the Stack Overflow survey (d).

W1-M09 — c. Know-what is (a) and (d) — recall. Know-how is (b) — building it. Know-why is choosing and justifying by cost, which is what the exam weighs.

W1-M10 — b. Both thresholds, from the bylaws: $\geq 60\%$ overall **and** $\geq 30\%$ of the final. (c) is the common misreading — a strong coursework mark cannot rescue a final below 30%. And attendance below 75% bars you from the final entirely.

W1-M11 — d. `//` is *floor* division: it rounds **down**. -3.5 rounded down is -4 . -3 (a) is rounding towards zero, which is C's behaviour; -3.5 (b) is what `/` gives.

W1-M12 — a. Python keeps `a == (a // b) * b + a % b`. Here $-7 // 3$ is -3 , and $-3 \times 3 + 2 = -7$, so the remainder is 2. With a positive divisor, `%` is never negative in Python.

W1-M13 — b. `**` groups right to left: $2 ** (3 ** 2) = 2^9 = 512$. 64 (a) is $(2 ** 3) ** 2$.

W1-M14 — c. `/` always returns a `float`, even when the division is exact: $8 / 4$ is `2.0`. Use `//` for an integer result.

W1-M15 — d. Indices 1, 2, 3 — up to but **not including** 4: `"yth"`. `"ytho"` (b) includes index 4.

W1-M16 — a. Step -2 from the end: indices 7, 5, 3, 1 $\rightarrow$ a, u, s, a. `"aruosnaM"` (b) is `[: :-1]`; `"Mnor"` (c) is `[: :2]`.

W1-M17 — c. `b = a` copies nothing: both names refer to the **same** list, so appending through `b` changes what `a` sees.

W1-M18 — b. `a[:]` makes a new list with equal contents: a different object (`is` is `False`) with the same values (`==` is `True`).

W1-M19 — c. `[[0] * 2] * 2` puts **two references to one inner list** into the outer list. Changing it through `grid[0]` shows through `grid[1]`. The correct construction is `[[0] * 2 for _ in range(2)]`.

W1-M20 — a. Python rounds exact halves to the nearest **even** number: `round(2.5)` is 2 and `round(3.5)` is 4 — total 6.

W1-M21 — d. A float is a binary fraction; 0.1 has no exact binary representation, so `0.1 + 0.2` is `0.30000000000000004`. Not a bug (a) — every language with IEEE floats does this. Compare floats with a tolerance.

W1-M22 — b. Strings are immutable. (c) is how you *should* do it — `s = "z" + s[1:]` — but item assignment itself is refused.

W1-M23 — c. A function with no return returns `None`. The 12 appears on the screen, but it is not *returned*. Tests see only what is returned.

W1-M24 — d. `int()` of a string accepts only an integer literal. Use `int(float("4.5"))` if truncation is really what you want.

Part B — Short answer and essay

Model answers. The **marking points** are what an examiner looks for; each is roughly one mark.

W1-E1 (4)

- **True part:** general-purpose languages are Turing-complete, so they can all compute the same set of functions.
- **What is wrong:** equivalence of *what* can be computed says nothing about the **cost** — to write, to read and maintain, to run — nor about which mistakes the language lets survive into production.
- A language is a **set of design decisions** with authors and reasons (language → design → philosophy).
- **Example**, any one well explained: C trusts the programmer and does no bounds checking, so `a[10]` on a 5-element array silently corrupts memory, while Python raises `IndexError`; or Python is strongly typed (`"3" + 5` is an error) while JavaScript is weak (`"35"`); or Rust's ownership versus C's manual free.

W1-E2 (4)

- **Static vs dynamic** is *when* types are checked — at compile time, or at run time.
- **Strong vs weak** is *how strictly* types are kept apart — refusing, or silently converting.
- Python is **dynamic and strong**; JavaScript is **dynamic and weak**.
- `"3" + 5`: Python raises `TypeError`; JavaScript returns `"35"`. Neither is a bug — they are different decisions about loud failure versus convenient conversion.

W1-E3 (3)

- **Manual** (C, C++): the programmer allocates and frees — fast, and the source of many security bugs. **Garbage collected** (Python, Java, Go): the runtime frees unreachable objects — safe, less control. **Ownership** (Rust): the compiler proves each allocation is freed exactly once — safe and fast, harder to satisfy.
- Python uses **garbage collection** (reference counting plus a cycle collector).
- Consequence: removing a node from a linked list is just "stop referring to it" — no free, no visible pointer — so the structure is invisible unless you use the debugger to watch references change.

W1-E4 (4)

- **ADT:** the contract — operations, their meaning, their cost — with nothing about storage.
- **Data structure:** a concrete arrangement of data in memory that fulfils the contract.
- **Stack ADT:** push adds, pop removes the most recent, peek looks; all $O(1)$.
- **Two structures:** a dynamic array with the top at the end; a linked list with the top at the head. Both honest stacks; they differ in memory overhead, cache behaviour and whether one operation can occasionally be slow. Choosing between them is the engineering.

W1-E5 (3)

- **For**, any two: the field (research, data, AI/ML) runs on Python; it binds to fast C/C++/CUDA underneath; it reads close to pseudocode, so the algorithm survives onto the page; students meet it again in every program.
- **Against**: Python already has the structures as built-ins — `list`, `dict`, `deque`, `sorted`, `heapq` — which hides exactly what the course studies.
- **Remedy**: the course reimplements them, and (from Lecture 02) structures may store data only in the course `Array`, in nodes, or in structures students built — never in `list`, `dict` or `set`.

W1-E6 (3)

- `a = [1, 2, 3]` creates a list object and binds the name `a` to it; `b = a` binds a **second name to the same object** — no copy.
 - Diagram: two names, `a` and `b`, with arrows to one list object.
 - Consequence: `b.append(4)` changes what `a` shows. To get an independent list, copy it: `a[:]`, `a.copy()` or `list(a)` — and that copy is **shallow**.
-

Part C — Trace the code**W1-T1**

```
3 2 -4 3
(5, 3) 1024 3.5
```

`-17 // 5` is `-3.4` rounded **down**, `-4`; and `-4 × 5 + 3 = -17`, so `-17 % 5` is `3`.

W1-T2

```
D s Stru DATA
['Data', 'Structures'] 15 3
```

`s[5:9]` is indices 5, 6, 7, 8 — `S t r u. split()` with no argument splits on runs of whitespace. The three `ts`: one in *Data*, two in *Structures*.

W1-T3

```
[99, 2, 3, 4] [99, 2, 3, 4] [1, 2, 3, 5]
[99, 2, 3, 4, 6]
```

`a` and `b` are one list, so `append` and `b[0] = 99` both show in both. `c` was a copy taken *before* those changes; `c + [5]` builds a new list. `a += [6]` extends the list **in place**, so `b` sees it — unlike `a = a + [6]`, which would have made a new list for `a` only.

W1-T4

```
1 6
```

`1000 → 333 → 111 → 37 → 12 → 4 → 1`: six divisions by 3. Dividing by 3 until 1 takes about $\log_3 n$ steps — the Week 2 idea of logarithmic cost.

W1-T5

```
Omar|87.5|    87.46|004
```

>6 right-aligns in 6 characters; .1f rounds to one decimal; 8.2f is 8 characters wide with two decimals; 03d pads the integer to 3 digits with zeros.

W1-T6

```
[0, 1, 1, 2, 3, 5, 8, 13]
[5, 8, 13] [0, 2, 8]
```

[::3] takes indices 0, 3, 6.

Part D — Find and fix the bug

W1-B1. `split(" ")` splits on **each single space**, so two spaces in a row, or a leading or trailing space, produce empty strings "", and ""[0] raises `IndexError`. Input that fails: "Ada Lovelace" or " Ada". **Fix:** use `split()` with no argument, which splits on runs of whitespace and drops empty pieces.

W1-B2. `i` is never incremented, so the loop never ends for any non-empty list — the program hangs (Ctrl+C to stop it). **Fix:** add `i += 1` inside the loop — or, better, for `v in values:` `total += v`, which cannot forget.

W1-B3. `result = values` does **not** copy; append then changes the caller's list, which the specification forbids. Test: after `with_item(original, 9)`, `original` has changed. **Fix:** `result = values[:]` (or `values.copy()`, or simply `return values + [item]`).

Part E — Write the code

Worked solutions. Yours may differ and still be correct — the tests decide.

W1-C1

```
def count_vowels(text):
    count = 0
    for ch in text.lower():
        if ch in "aeiou":
            count += 1
    return count
```

W1-C2 — one pass: keep the largest and the second largest seen so far.

```
def second_largest(values):
    largest = second = None
    for v in values:
```

```

    if largest is None or v > largest:
        if largest is not None:
            second = largest          # the old maximum is now second
            largest = v
        elif v != largest and (second is None or v > second):
            second = v
    if second is None:
        raise ValueError("need at least two distinct values")
    return second

```

The `v != largest` check is what makes `[9, 9, 7]` give 7, not 9. Sorting would work too but is $O(n \log n)$; this is $O(n)$.

W1-C3 — find where each run ends.

```

def compress(text):
    result = ""
    i = 0
    while i < len(text):
        j = i
        while j < len(text) and text[j] == text[i]:
            j += 1          # j stops at the end of the run
        result += text[i] + str(j - i)
        i = j
    return result

```

Each character is looked at a constant number of times, so this is $O(n)$ steps (ignoring the cost of building the string).

W1-C4

```

def is_anagram(first, second):
    a = sorted(first.replace(" ", "").lower())
    b = sorted(second.replace(" ", "").lower())
    return a == b

```

$O(n \log n)$ because of sorting. Counting letters in a dictionary (Lab 03) makes it $O(n)$.